

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT /
SUMMARIZER / TRANSLATOR)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 1-Mini Application Development (Chatbot / Summarizer / Translator)
Weightage:	20% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	A. Mounish	Reg. No.:	192472273
Section / Batch:	CSE AI	Date:	20/08/26

Instructions to Students

- **Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.**
- Implement the solution using **Python**.
- Select and justify an appropriate AI/LLM model or technique.
- Develop a **working application with a user interface**.
- Test the application using multiple input cases.
- Demonstrate handling of invalid, empty, or unexpected inputs.

- Clearly explain the application architecture and workflow.
- Submit the source code, test results, screenshots/demo, and documentation.
- Explain the limitations of the selected model/technique and suggest possible improvements.

Code of Conduct

I A. Mounish / 192472273 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

SECTION A – MINI APPLICATION DEVELOPMENT (CHATBOT / SUMMARIZER / TRANSLATOR)

AI CHATBOT APPLICATIONS

7. Examination Support Chatbot – Build a chatbot that answers student queries about examination procedures, examination rules, registration, hall tickets, internal assessments, and result-related information.

1. Question Requirement

Question 7 requires the development of an Examination Support Chatbot that answers student queries related to examination procedures, examination rules, registration, hall tickets, internal assessments, and result-related information.

The assessment instructions require a Python implementation, an appropriate AI/LLM model or technique, a working user interface, multiple input tests, handling of invalid or empty inputs, an explanation of architecture and workflow, source code and test evidence, and a discussion of limitations and possible improvements.

2. Abstract

The Examination Support Chatbot is a Python-based mini application designed to provide quick and consistent responses to common student examination queries. The system uses a lightweight Natural Language Processing technique based on Term Frequency–Inverse Document Frequency (TF-IDF) and Cosine Similarity. A small examination knowledge base stores questions and corresponding answers covering registration, examination rules, hall tickets, internal assessments, results, and related procedures.

A Streamlit interface provides a simple and user-friendly way for students to enter questions and receive responses. The application also includes validation for empty input and a fallback mechanism for questions that are outside the available examination knowledge. The implemented system was tested using multiple question types, and the supplied screenshots demonstrate successful responses for the major functional areas.

3. Introduction

Students frequently need clarification about examination procedures, registration, hall tickets, internal assessment rules, and result-related processes. Repeatedly asking the same basic questions to staff can consume time and may lead to inconsistent communication. A chatbot can provide a first level of assistance by presenting prepared examination information through a conversational interface.

The proposed application focuses on a clearly defined examination-support domain. Instead of attempting to answer unrestricted questions, it compares the student's query with a curated set of examination questions and selects the most relevant answer. This controlled approach makes the mini application simple to implement, easy to test, and suitable for a demonstration environment.

4. Objectives

- Develop a working examination-support chatbot using Python.
- Provide responses for examination registration, rules, hall tickets, internal assessments, and results.
- Use TF-IDF and Cosine Similarity to identify the most relevant knowledge-base question.
- Provide a simple Streamlit graphical user interface.
- Handle empty questions safely without causing an application failure.
- Provide a fallback response for unrelated or unsupported questions.

- Test the application using multiple representative input cases.
- Document the architecture, workflow, limitations, and possible improvements.

5. Problem Identification

Students may have difficulty locating examination information quickly, especially when information is distributed across notices, portals, departments, or administrative instructions. A focused chatbot can act as a convenient first point of support. The problem addressed by this application is therefore the quick retrieval of relevant examination guidance from a predefined knowledge base through natural language questions.

6. Proposed Solution

The proposed solution consists of three major layers: the Streamlit user interface, the chatbot processing layer, and the examination knowledge base. The student enters a question in the interface. The chatbot converts the question into a TF-IDF representation, compares it with stored questions using Cosine Similarity, identifies the best matching question, and returns its corresponding answer when the similarity is sufficient. If the input is empty, the interface displays a validation warning. If the question is unrelated to the available knowledge, the chatbot returns a controlled fallback message.

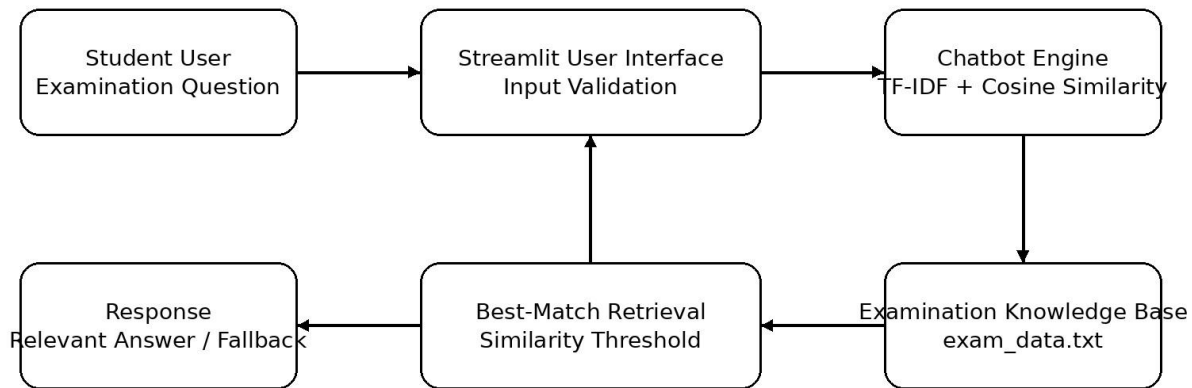
7. Model / Technique Used

The implemented technique is TF-IDF with Cosine Similarity. TF-IDF represents text numerically by considering the importance of words within the available question set. Cosine Similarity then measures how closely the student's query matches each stored question. The question with the highest similarity score is selected, subject to a minimum similarity threshold.

This technique was selected because the application has a small and focused knowledge base. It is lightweight, transparent, fast, and does not require a large model or external API for the demonstrated use case. The technique also allows the fallback threshold to be explicitly controlled, which is useful for preventing unrelated questions from receiving an arbitrary examination answer.

8. System Architecture

Examination Support Chatbot - Architecture & Workflow



Empty input → validation warning | Unrelated input → fallback response

Figure 1. Architecture and workflow of the Examination Support Chatbot.

The architecture begins with the student question and passes it through the Streamlit interface. The chatbot engine uses TF-IDF vectorization and Cosine Similarity to compare the input against the examination question set. The selected response is returned to the interface. The knowledge base is maintained separately in `exam_data.txt`, which makes the examination content easy to update without changing the main interface.

9. Workflow

- Step 1: The application starts and loads the examination knowledge base.
- Step 2: Stored questions are converted into TF-IDF vectors.
- Step 3: The student enters an examination-related question.
- Step 4: The application checks whether the input is empty.
- Step 5: If the input is valid, it is converted into the same TF-IDF representation.
- Step 6: Cosine Similarity is calculated between the input and stored questions.
- Step 7: The question with the highest similarity is identified.
- Step 8: If the similarity exceeds the defined threshold, the corresponding answer is displayed.
- Step 9: If the similarity is too low, a fallback response is displayed.
- Step 10: The student can submit another question through the same interface.

10. Implementation Details

10.1 Knowledge Base – exam_data.txt

The knowledge base contains examination-related question-and-answer pairs. The implemented content covers registration procedures, examination rules, hall tickets, internal assessments, result checking, result problems, examination-hall requirements, and general examination guidance. This provides the information used by the retrieval component.

10.2 Chatbot Engine – chatbot.py

The chatbot module loads the question-and-answer pairs, creates a TfidfVectorizer, and transforms the stored questions into vectors. When the user submits a question, the module transforms the input into a vector and calculates Cosine Similarity against all stored question vectors. The highest-scoring answer is returned when the score is at least 0.20; otherwise, a fallback message is returned.

The module also checks for empty input before performing vectorization. This prevents invalid input from causing an unnecessary processing error.

10.3 User Interface – app.py

The user interface is implemented using Streamlit. It displays the title, purpose of the chatbot, a text input field, an Ask Chatbot button, and a response area. The interface gives a clear warning when no question is entered and displays the chatbot answer when a valid question is submitted.

11. User Interface and Demonstration

The following screenshots are the actual demonstration evidence supplied during implementation. They show the working Streamlit application and responses for different examination-support questions.



Examination Support Chatbot

Ask questions about examination procedures, rules, registration, hall tickets, internal assessments, and results.

Enter your examination question:

How can I download my hall ticket?

Ask Chatbot

Chatbot Response

Log in to the examination portal and open the hall-ticket section. Verify the details and download or print the hall ticket.

AI technique: TF-IDF with Cosine Similarity

Figure 2. Working chatbot interface with examination registration response.

This screenshot demonstrates that the chatbot can identify a registration-related query and return the corresponding examination registration procedure.



Examination Support Chatbot

Ask questions about examination procedures, rules, registration, hall tickets, internal assessments, and results.

Enter your examination question:

How can I register for the examination?

Ask Chatbot

Chatbot Response

Open the examination portal, log in with your student credentials, select the examinations or subjects, verify the details, and submit the registration.

Figure 3. Examination rules query and response.



Examination Support Chatbot

Ask questions about examination procedures, rules, registration, hall tickets, internal assessments, and results.

Enter your examination question:

What are the examination rules?

Ask Chatbot

Chatbot Response

Students must follow the examination rules, carry their valid hall ticket and college ID, arrive on time, and follow the instructions given by the invigilator.

Figure 4. Hall-ticket download query and response.

This screenshot demonstrates retrieval of the hall-ticket download procedure from the examination knowledge base.



Examination Support Chatbot

Ask questions about examination procedures, rules, registration, hall tickets, internal assessments, and results.

Enter your examination question:

How can I download my hall ticket?

Ask Chatbot

Chatbot Response

Log in to the examination portal and open the hall-ticket section. Verify the details and download or print the hall ticket.

Figure 5. Hall-ticket definition query and response.



Examination Support Chatbot

Ask questions about examination procedures, rules, registration, hall tickets, internal assessments, and results.

Enter your examination question:

What is a hall ticket?

Ask Chatbot

Chatbot Response

A hall ticket is an examination document containing important details such as the student's name, registration number, examination, date, time, and examination hall.

Figure 6. Internal assessment query and response.

This screenshot demonstrates support for internal-assessment questions, one of the specific functional areas required by Q7.



Examination Support Chatbot

Ask questions about examination procedures, rules, registration, hall tickets, internal assessments, and results.

Enter your examination question:

What are internal assessments?

Ask Chatbot

Chatbot Response

Internal assessments are evaluations conducted during the academic period. They may include tests, assignments, activities, or other academic evaluations.

Figure 7. Internal assessment marks query and response.



Examination Support Chatbot

Ask questions about examination procedures, rules, registration, hall tickets, internal assessments, and results.

Enter your examination question:

How are internal assessment marks calculated?

Ask Chatbot

Chatbot Response

Internal assessment marks are calculated according to the assessment methods and rules specified by the institution or course.

Figure 8. Examination result query and response.

This screenshot demonstrates result-related question handling through the same retrieval workflow.



Examination Support Chatbot

Ask questions about examination procedures, rules, registration, hall tickets, internal assessments, and results.

Enter your examination question:

How can I check my examination result?

Ask Chatbot

Chatbot Response

Visit the official result portal, enter the required student credentials or registration details, and view the published result.

Figure 9. Result-not-displayed problem handling.



Examination Support Chatbot

Ask questions about examination procedures, rules, registration, hall tickets, internal assessments, and results.

Enter your examination question:

What if my result is not displayed?

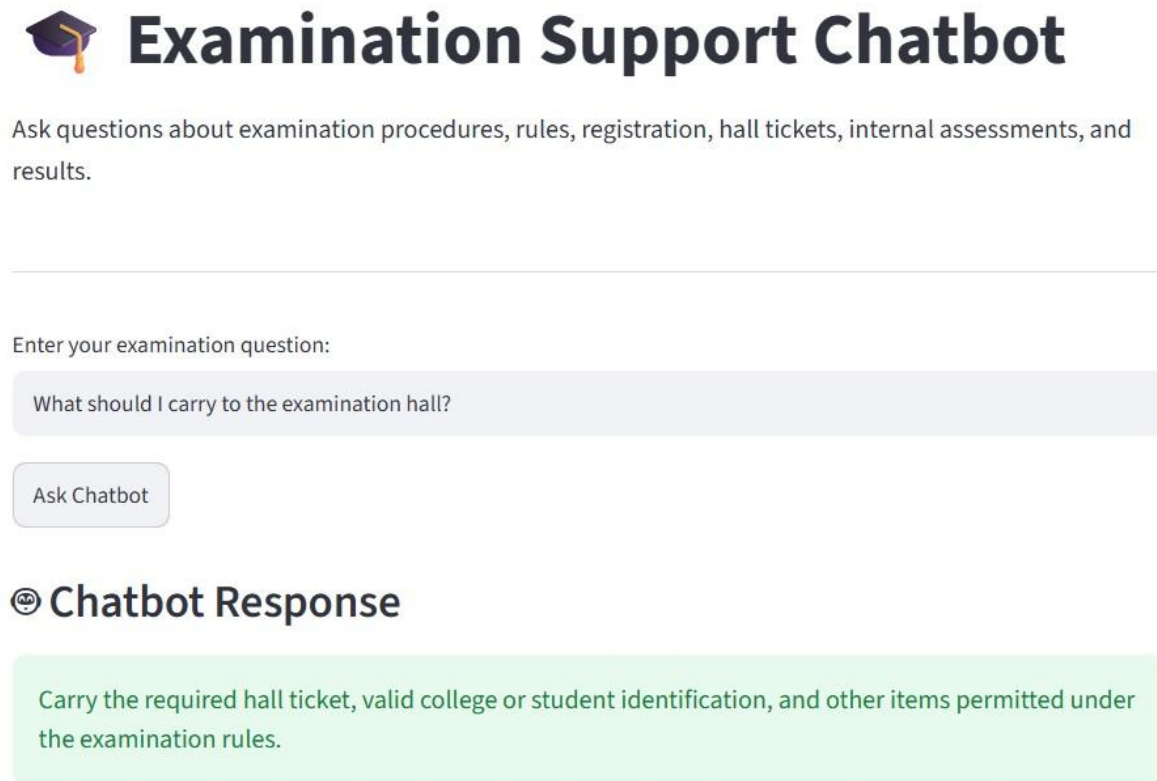
Ask Chatbot

Chatbot Response

Verify your registration details and try again later. If the problem continues, contact the concerned examination authority.

Figure 10. Examination-hall requirements query and response.

This screenshot demonstrates support for examination-hall requirements.



The screenshot shows the 'Examination Support Chatbot' interface. At the top, there is a header with a graduation cap icon and the title 'Examination Support Chatbot'. Below the header, a subtitle reads: 'Ask questions about examination procedures, rules, registration, hall tickets, internal assessments, and results.' A horizontal line separates the header from the input area. The input area contains the prompt 'Enter your examination question:' followed by a text input field with the text 'What should I carry to the examination hall?'. Below the input field is a button labeled 'Ask Chatbot'. The response area, titled 'Chatbot Response' with a speech bubble icon, contains a green box with the text: 'Carry the required hall ticket, valid college or student identification, and other items permitted under the examination rules.'

Figure 11. Unrelated question handled using fallback response.

This screenshot demonstrates robustness through a controlled fallback for an unrelated question.

12. Input Validation and Robustness

Robustness is demonstrated through two explicit handling mechanisms. First, when the student submits an empty input, the interface displays 'Please enter a question.' rather than attempting to process an empty query. Second, when the student enters a question outside the examination knowledge domain, the chatbot does not force an unrelated answer. Instead, it returns a fallback message asking the user to ask about supported examination topics.

These mechanisms are important because the assessment instructions specifically require demonstration of invalid, empty, or unexpected input handling.

13. Test Cases and Results

Test ID	Input	Expected Behavior	Result
TC01	How can I register for the examination?	Registration procedure	Pass

TC02	What are the examination rules?	Examination rules	Pass
TC03	How can I download my hall ticket?	Hall-ticket procedure	Pass
TC04	What is a hall ticket?	Hall-ticket explanation	Pass
TC05	What are internal assessments?	Internal assessment information	Pass
TC06	How are internal assessment marks calculated?	Assessment marks information	Pass
TC07	How can I check my examination result?	Result checking procedure	Pass
TC08	What if my result is not displayed?	Result issue handling	Pass
TC09	What should I carry to the examination hall?	Examination-hall requirements	Pass
TC10	Tell me about tomorrow's weather	Fallback response	Pass
TC11		Empty-input warning	Pass

Table 1. Functional and robustness test cases. The results are based on the supplied implementation screenshots and completed tests.

14. Test Result Summary

All 11 documented test cases passed during the demonstration. The functional tests covered the main Q7 information areas, while the final two cases verified fallback and empty-input handling. The successful demonstrations show that the implemented application operates correctly for the tested scenarios.

15. Functional Features

- Examination registration support.
- Examination rules support.
- Hall-ticket information and download guidance.
- Internal assessment information.
- Internal assessment marks guidance.
- Examination result checking guidance.
- Result problem guidance.
- Examination-hall requirements.
- Empty-input validation.
- Fallback response for unsupported questions.
- Simple browser-based Streamlit interface.

16. Advantages

- Simple and easy-to-use interface.
- Fast response for supported examination queries.
- Lightweight implementation without an external API dependency.
- Easy-to-update text knowledge base.
- Controlled fallback for unsupported questions.
- Clear separation between interface, chatbot logic, and knowledge data.

17. Limitations

- The system can answer only questions represented adequately by the available examination knowledge base.
- TF-IDF and Cosine Similarity do not provide the deep semantic understanding of a modern large language model.
- The application does not automatically retrieve live examination notices or result information from an institutional portal.
- The current knowledge base is manually prepared and therefore must be updated when examination procedures change.
- The chatbot does not maintain a multi-turn conversational memory.

18. Future Improvements

- Replace or supplement the retrieval technique with a modern embedding model for stronger semantic matching.
- Integrate Retrieval-Augmented Generation (RAG) with official examination documents.
- Connect to authorized institutional data sources for current examination schedules and notices.
- Add multilingual support for English and Indian languages.
- Add conversation history and context-aware follow-up questions.
- Add authentication where student-specific information is required.
- Add document upload and PDF-based examination information retrieval.

19. Conclusion

The Examination Support Chatbot successfully implements the core requirements of Question 7. It provides a working Python and Streamlit application that responds to examination-related questions using TF-IDF and Cosine Similarity. The application covers examination registration, rules, hall tickets, internal assessments, and results, while also handling empty and unrelated inputs through validation and fallback responses.

The supplied screenshots provide direct evidence of the working user interface and successful test cases. The system is lightweight and suitable as a focused mini application, while the proposed future improvements provide a clear path toward a more advanced RAG- or LLM-based examination assistant.

20. Requirements Mapping

Assessment Requirement	Evidence in Report
Python implementation	Completed using app.py and chatbot.py.
Appropriate AI/LLM technique	TF-IDF + Cosine Similarity selected for focused question retrieval.
Working user interface	Streamlit browser interface implemented and demonstrated.
Multiple input cases	11 test cases documented, including major examination topics.
Invalid/empty/unexpected inputs	Empty-input warning and unrelated-query fallback demonstrated.
Architecture and workflow	Architecture diagram and step-by-step workflow included.
Source code / test evidence / screenshots	Implementation structure and supplied screenshots documented.
Limitations and improvements	Dedicated limitations and future-improvement sections included.

21. References

1. Assessment Tool 1 – Mini Application Development (Chatbot / Summarizer / Translator), SIMATS Engineering, Department of Computer Science and Engineering, supplied course PDF.
2. Scikit-learn documentation concepts used in the implementation: TF-IDF vectorization and cosine similarity.
3. Streamlit documentation concepts used in the implementation: text input, button controls, messages, and application configuration.